

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Mon 30 AUGUST 2021 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.00m

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R below, over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$R = a (b \mid c^+ a)^*$$

Answer the following questions:

- (a) Write all the strings x of the language of R that have a length less than or equal to four, i.e., $x \in L(R)$ with $|x| \leq 4$, in lexicographic order (with $a < b < c$).
 - (b) By means of the Berry-Sethi (BS) method, find a deterministic automaton A equivalent to the regular expression R .
 - (c) Is the deterministic automaton A found before minimal ? Justify your answer.
 - (d) By means of the Brzozowski method (node elimination), starting from the automaton A found before, obtain a regular expression R' equivalent to A .
 - (e) (optional) Is the language $L(R)$ locally testable ? Formally prove your answer.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the language L_1 below, over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$L_1 = \{ a^m b c^n \mid m > n > 0 \} \cup \{ c^n b a^m \mid m > n > 0 \}$$

Examples and counterexamples:

$$a^3 b c^2, c b a^2 \in L_1$$

$$a^2 b c^3, c^3 b a^2 \notin L_1$$

Answer the following questions:

- (a) Write a *BNF* grammar G_1 , *not extended*, that generates language L_1 .
- (b) Say if your grammar G_1 is ambiguous, and suitably motivate your answer.
- (c) Draw the syntax tree, or all the trees in case of ambiguity, for the string below:

$$c^2 b a^3 \in L_1$$

Next consider the language L_2 , over the same alphabet Σ , the strings of which consist of an even number $k \geq 4$ of non-empty groups of letters a and of letters c . The groups alternate and start with letters a , such that:

- the groups of letters a and the groups of letters c are separated by $k - 1$ isolated letters b
- the groups of letters a and the groups of letters c that are located in a symmetric position with respect to the central letter b , have the same length

Examples and counterexamples:

$$a^3 b c^2 b a^2 b c^3, a b c b a b c b a b c \in L_2$$

$$a^3 b c^2 b a^3, a^3 b c^2 b a^3 b c^2 \notin L_2$$

Answer the following questions:

- (d) Write a *BNF* grammar G_2 , *not extended and not ambiguous*, that generates language L_2 .
- (e) Draw the syntax trees for the two strings below:

$$a b c^2 b a^2 b c, a b c b a b c b a b c \in L_2$$

2. We intend to model the *Boolean* expressions of an Algol-like programming language. Such expressions feature the operators below, listed in order of decreasing precedence:

- $<$ and $>$: relational operators, binary, applicable only to arithmetic expressions
- **not**: negation, unary
- **and**: conjunction, binary, right-associative
- **or**: disjunction, binary, right-associative
- **impl**: implication, binary, right-associative
- **if ... then ... else ... fi**: conditional

All the above logical operators (except for the relational operators) are applicable only to Boolean expressions.

Examples:

$v1 > z3$	– relational operator
$x > z$ or $z < y$	– logical and relational operators
r and (s impl $x > y$)	– parentheses are allowed
if if not r then q else f fi then s else t fi	– structure if can be nested

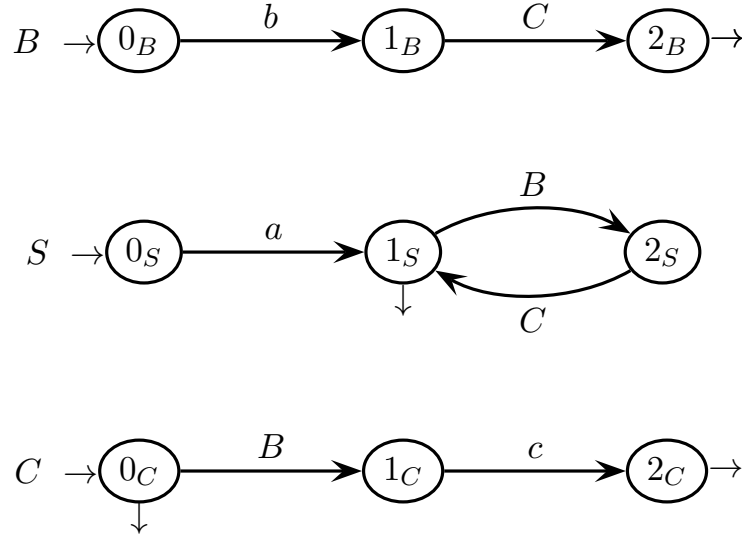
Answer the following questions:

- (a) Write a grammar, *BNF*, *not extended and not ambiguous*, that defines the Boolean expression language outlined above. Boolean atomic expressions (constants and variables) are represented by the terminal symbol **b**. Arithmetic expressions are represented by the terminal symbol **a**.
- (b) Draw the syntax tree for the Boolean expression below:

r and (s impl $x > y$)

3 Syntax Analysis and Parsing Methodologies 20%

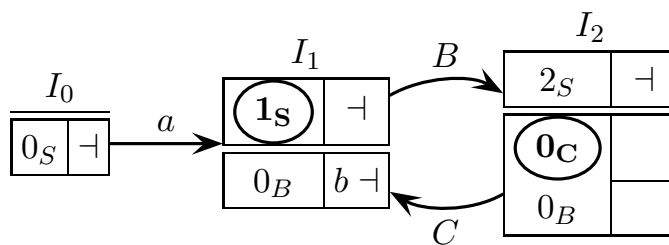
1. Consider the *EBNF* grammar G below, presented as a net of machines, over the nonterminal and terminal alphabets $V = \{ B, C, S \}$ and $\Sigma = \{ a, b, c \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

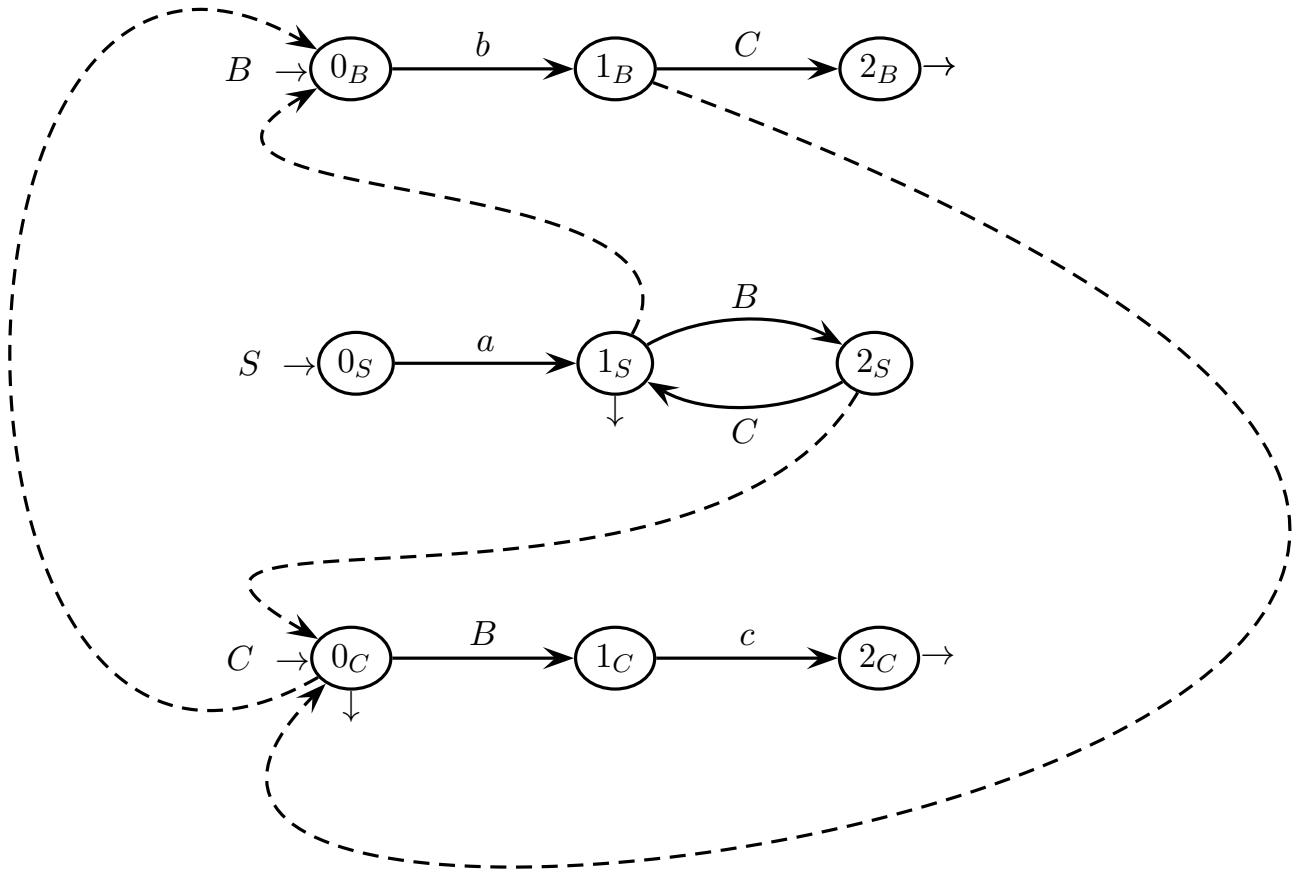
- (a) Complete the graph of the ELR pilot.
 - (b) Show that grammar G is not of type $ELR(1)$ and list all the pilot conflicts.
 - (c) Complete the machine net with the guide sets on the call arcs and exit arrows.
 - (d) Show that grammar G is not of type $ELL(1)$ and list all the guide set conflicts.
 - (e) Is the language $L(G)$ regular ? Justify your answer.
 - (f) (optional) As machine M_B does not have any bifurcation points, such a machine is of type $ELL(1)$ independently of the other machines. Write the recursive descent syntactic procedure of machine M_B .
-

question (a) – pilot graph to be completed



question (b) – list of pilot conflicts

question (c) – machine net to be completed with guide sets



question (d) – list of guide set conflicts

question (e) – nature of language $L(G)$

question (f) – syntactic procedure of machine M_B

4 Language Translation and Semantic Analysis 20%

1. The *BNF* grammar G below (axiom S):

$$G: S \rightarrow o S c S \mid \varepsilon$$

defines the Dyck language with brackets “ o ” (open) and “ c ” (closed). We intend to define a translation scheme τ , where the bracket pairs placed at an **odd** nesting level are turned into pairs “ a ” and “ b ”, while those placed at an **even** nesting level are turned into pairs “ d ” and “ e ”. Notice that the outermost brackets are considered at level 1 (see also the example).

Here is a sample translation (spacing visually helps identifying the bracket pairs):

$$\begin{aligned} s &= o o c c o o c o o c c c \\ \tau(s) &= a d e b a d e d a b e b \end{aligned}$$

Answer the following questions:

- (a) Modify the above grammar G so as to obtain the *BNF* source grammar G_s of a translation scheme suitable for the definition of the translation τ specified above.
- (b) Complete the translation scheme with the *BNF* target (destination) grammar G_t that matches the source grammar G_s defined for the previous question.
- (c) Draw the source and target (destination) syntax trees for the source string:

$$s = o c o o o c c c$$

2. We intend to translate an arithmetic expression from the infix form to the postfix or prefix forms, by using an attribute grammar instead of a syntactic translation scheme. We refer to the following simplified grammar G (axiom S) as syntactic support. Grammar G features only right associative binary addition “+”, round brackets “(” and “)”, and atomic expressions (constants and variables) represented by terminal symbol a :

$$G \left\{ \begin{array}{l} S \rightarrow E \\ E \rightarrow T + E \\ E \rightarrow T \\ T \rightarrow (E) \\ T \rightarrow a \end{array} \right.$$

A first translation τ_1 simply produces the usual postfix form and eliminates the brackets. For instance:

$$\tau_1 (a +_1 (a +_2 a) +_3 a) = a a a +_2 a +_3 +_1$$

where the subscripts are added to the operator “+” only for the sake of readability.

Answer the following questions (use the figures / tables / spaces on the next pages):

- Write an attribute grammar, with grammar G as syntactic support, that computes translation τ_1 (without subscripts on the operator “+”) by using only a string *left* attribute s associated with the nonterminals S , E and T . Translation τ_1 is computed as the attribute s of the root node S .
- Decorate the tree (see the next page) of the source string “ $a +_1 (a +_2 a) +_3 a$ ” by applying the values of attribute s at each nonterminal node.

A second translation τ_2 keeps the round brackets, produces the postfix form for the (sub)expressions that are enclosed in an *even* number (including zero) of bracket pairs, and produces the *prefix* form for the (sub)expressions that are enclosed in an *odd* number of bracket pairs. For instance:

$$\tau_2 (a +_1 (a +_2 a) +_3 a) = a (+_2 a a) a +_3 +_1$$

$$\tau_2 (a +_1 (a +_2 ((a +_3 a) +_4 a)) +_5 a) = a(+_2 a((+_3 a a) a +_4)) a +_5 +_1$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- Write an attribute grammar, with grammar G as syntactic support, that computes translation τ_2 (without subscripts on the operator “+”) by using two attributes: a boolean *right* attribute e associated with nonterminals E and T , and a string *left* attribute s associated with nonterminals S , E and T . Attribute e means that the current (sub)expression is enclosed in an even number of bracket pairs. Translation τ_2 is computed as the attribute s of the root node S .
- Decorate the tree (in the second next page) of the source string “ $a +_1 (a +_2 a) +_3 a$ ” by applying the values of attributes e and s at each nonterminal node.

syntax

semantics for translation τ_1 – question (a)

1: $S_0 \rightarrow E_1$

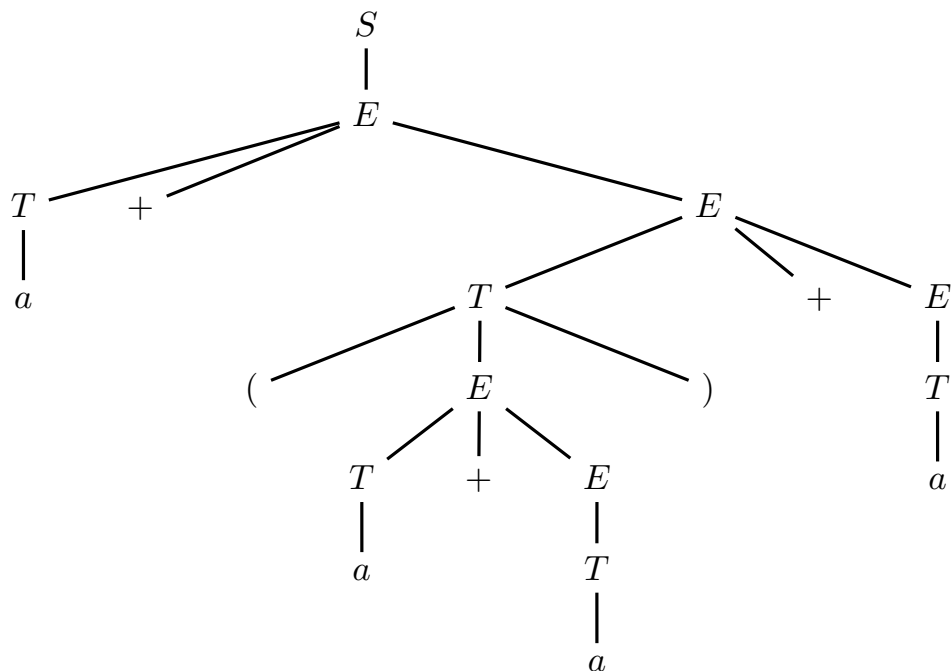
2: $E_0 \rightarrow T_1 + E_2$

3: $E_0 \rightarrow T_1$

4: $T_0 \rightarrow (E_1)$

5: $T_0 \rightarrow a$

question (b) – tree to be decorated



syntax

semantics for translation τ_2 – question (c)

1: $S_0 \rightarrow E_1$

2: $E_0 \rightarrow T_1 + E_2$

3: $E_0 \rightarrow T_1$

4: $T_0 \rightarrow (E_1)$

5: $T_0 \rightarrow a$

question (d) – tree to be decorated

